[OLC2] Tarea 3 – Código de 3 Direcciones con ANTLRv4

Universidad San Carlos de Guatemala
Facultad de Ingeniería – Ingeniería en Ciencias y Sistemas
Organización de Lenguajes y Compiladores 2 – 1er Semestre 2026
Ebed Isai Patzan Tzic — 202308204

Estructura del Proyecto

```
tarea3/
├── Grammar.g4          ← Gramática ANTLR4 extendida con &&, ||, if sin
paréntesis
├── C3DGenerator.php    ← Clase para manejar temporales, etiquetas e
instrucciones
├── C3DVisitor.php      ← Visitor que recorre el árbol y genera el C3D con
cortocircuito
└── index.php           ← Punto de entrada / pruebas
```

Parte 1 – Estructura de Control If-Else con Operadores Lógicos

Gramática ANTLR4 extendida (`Grammar.g4`)

Cambios respecto a la Tarea 2:

Aspecto	Tarea 2	Tarea 3
Sintaxis if	`if (condición) {`	`if condición {` (sin paréntesis obligatorios)
Condición lógica	Solo `relOp` simple	`&&` y `||` con cortocircuito
print	Solo `print(ID)`	`print(ID)` o `print("string")`
Mid-label	No existía	Necesario para cortocircuito

```
grammar Grammar;

program
    : statement+ EOF
    ;

statement
    : assignment SEMICOLON?     # StmtAssign
    | printStmt  SEMICOLON?     # StmtPrint
    | ifStmt                    # StmtIf
```

```
    ;

assignment
    : ID ASSIGN expr
    ;

printStmt
    : PRINT LPAREN (ID | STRING) RPAREN
    ;

ifStmt
    : IF condition LBRACE statement+ RBRACE
      ( ELSE LBRACE statement+ RBRACE )?
    ;

// Condición con cortocircuito: || menor precedencia que &&
condition
    : condition OR  condition   # CondOr
    | condition AND condition   # CondAnd
    | expr relOp expr           # CondRelOp
    | LPAREN condition RPAREN   # CondParen
    ;

relOp
    : GT | LT | GEQ | LEQ | EQ | NEQ
    ;

expr
    : expr op=(MUL | DIV) expr  # ExprMulDiv
    | expr op=(ADD | SUB) expr  # ExprAddSub
    | LPAREN expr RPAREN        # ExprParen
    | NUMBER                    # ExprNumber
    | ID                        # ExprId
    ;

IF:'if'; ELSE:'else'; PRINT:'print';
AND:'&&'; OR:'||';
ADD:'+'; SUB:'-'; MUL:'*'; DIV:'/';
GEQ:'>='; LEQ:'<='; GT:'>'; LT:'<'; EQ:'=='; NEQ:'!=';
ASSIGN:':=';
LPAREN:'('; RPAREN:')'; LBRACE:'{'; RBRACE:'}'; SEMICOLON:';';
NUMBER : [0-9]+ ('.' [0-9]+)? ;
ID     : [a-zA-Z_][a-zA-Z_0-9]* ;
STRING : '"' ~["\r\n]* '"' ;
WS     : [ \t\r\n]+ -> skip ;
COMMENT: '//' ~[\r\n]* -> skip ;
```

Clase `C3DGenerator.php`

Igual que en la Tarea 2. Gestiona el estado global de la generación:

Método	Descripción
`newTemp(): string`	Genera un nuevo temporal `t1, t2, t3 …`
`newLabel(): string`	Genera una nueva etiqueta `L1, L2, L3 …`
`emit(string $inst)`	Agrega una instrucción al listado C3D
`printC3D()`	Imprime todas las instrucciones generadas

Clase `C3DVisitor.php`

Visitor con método `visitConditionWithLabels` para cortocircuito:

Método Visitor	Regla Grammar.g4	Acción
`visitProgram`	`program`	Itera sobre todas las sentencias
`visitAssignment`	`StmtAssign`	Evalúa expr → emite `ID = tN`
`visitPrintStmt`	`StmtPrint`	Emite `print ID` o `print "string"`
`visitIfStmt`	`StmtIf`	Genera etiquetas y delega condición con cortocircuito
`visitConditionWithLabels`	`CondAnd / CondOr / CondRelOp`	Cortocircuito real pasando etiquetas destino
`visitExpr*`	`ExprMulDiv / ExprAddSub`	Emite `tN = op1 oper op2`, retorna `tN`
`visitExprParen`	`ExprParen`	Delega al `inner` sin instrucción extra
`visitExprNumber / visitExprId`	`ExprNumber / ExprId`	Retorna el literal o nombre directamente

Algoritmo de cortocircuito

AND (`&&`):

```
Evalúa LEFT  → si es FALSE → goto falseLabel  (cortocircuito, RIGHT no se
evalúa)
[midLabel:]
Evalúa RIGHT → si es TRUE  → goto trueLabel
               si es FALSE  → goto falseLabel
```

OR (`||`):

```
Evalúa LEFT  → si es TRUE  → goto trueLabel   (cortocircuito, RIGHT no se
evalúa)
[midLabel:]
```

```
Evalúa RIGHT → si es TRUE  → goto trueLabel
               si es FALSE  → goto falseLabel
```

Caso 1 — `if 8 > 7 && 9 > 3`

Entrada

```
if 8 > 7 && 9 > 3 {
    print("Verdadero")
} else {
    print("Falso")
}
```

Árbol Sintáctico (AST) – Caso 1

```
program
└── StmtIf
    └── ifStmt
        ├── condition  (CondAnd)
        │   ├── condition  (CondRelOp)
        │   │   ├── ExprNumber  8
        │   │   ├── relOp       >
        │   │   └── ExprNumber  7
        │   └── condition  (CondRelOp)
        │       ├── ExprNumber  9
        │       ├── relOp       >
        │       └── ExprNumber  3
        ├── trueStmts
        │   └── StmtPrint  print("Verdadero")
        └── elseStmts
            └── StmtPrint  print("Falso")
```

C3D Generado – Caso 1

```
t1 = 8 > 7
if t1 goto L4
goto L2
L4:
    t2 = 9 > 3
    if t2 goto L1
    goto L2
L1:
    print "Verdadero"
    goto L3
L2:
```

```
    print "Falso"
L3:
```

Tabla de temporales y etiquetas – Caso 1

Símbolo	Tipo	Significado
`t1`	Temporal	Resultado de `8 > 7` → 1 (true)
`t2`	Temporal	Resultado de `9 > 3` → 1 (true)
`L1`	Etiqueta	Inicio bloque **verdadero** (if)
`L2`	Etiqueta	Inicio bloque **falso** (else)
`L3`	Etiqueta	Punto de **reunión** (end)
`L4`	Etiqueta	Mid-label del AND: separa evaluación de ambos lados
`if t1 goto L4`	Salto cond.	Si LEFT es true, continúa evaluando RIGHT en L4
`goto L2`	Salto incond.	Cortocircuito: LEFT fue false, va directo al else
`if t2 goto L1`	Salto cond.	Si RIGHT es true, va al bloque verdadero
`goto L3`	Salto incond.	Sale del bloque true sin ejecutar el else

Flujo de ejecución (runtime) – Caso 1

1. `t1 = 8 > 7` → `t1 = 1` (true)
2. `if t1 goto L4` → salta a L4 (no hay cortocircuito, LEFT fue true)
3. `t2 = 9 > 3` → `t2 = 1` (true)
4. `if t2 goto L1` → salta a L1
5. `print "Verdadero"` ✓
6. `goto L3` → fin

Captura 1 – Ejecución `php index.php` (Caso 1)

```
isai@IsaiPatzan-deskop:~/Documentos/Github/dragon_examples_antlr4_php/C3D/Tarea3$ php
"/home/isai/Documentos/Github/dragon_examples_antlr4_php/C3D/Tarea3/index.php"
============================================================
CASO 1 — if-else con condición AND (&&) y cortocircuito
Entrada:
    if 8 > 7 && 9 > 3 {
        print("Verdadero")
    } else {
        print("Falso")
    }
============================================================

========== Código de 3 Direcciones (C3D) ==========

    t1 = 8 > 7
    if t1 goto L4
    goto L2
L4:
    t2 = 9 > 3
    if t2 goto L1
    goto L2
L1:
    print "Verdadero"
    goto L3
L2:
    print "Falso"
L3:

=====================================================
```

Caso 2 — `if 8 < 7 || 9 > 3`

Entrada

```
if 8 < 7 || 9 > 3 {
    print("Verdadero")
} else {
    print("Falso")
}
```

Árbol Sintáctico (AST) – Caso 2

```
program
└── StmtIf
    └── ifStmt
        ├── condition  (CondOr)
        │   ├── condition  (CondRelOp)
        │   │   ├── ExprNumber  8
        │   │   ├── relOp       <
        │   │   └── ExprNumber  7
        │   └── condition  (CondRelOp)
```

```
          │       ├── ExprNumber  9
          │       ├── relOp       >
          │       └── ExprNumber  3
          ├── trueStmts
          │   └── StmtPrint  print("Verdadero")
          └── elseStmts
              └── StmtPrint  print("Falso")
```

C3D Generado – Caso 2

```
t1 = 8 < 7
if t1 goto L1
goto L4
L4:
    t2 = 9 > 3
    if t2 goto L1
    goto L2
L1:
    print "Verdadero"
    goto L3
L2:
    print "Falso"
L3:
```

Tabla de temporales y etiquetas – Caso 2

Símbolo	Tipo	Significado
`t1`	Temporal	Resultado de `8 < 7` → 0 (false)
`t2`	Temporal	Resultado de `9 > 3` → 1 (true)
`L1`	Etiqueta	Inicio bloque **verdadero** (if)
`L2`	Etiqueta	Inicio bloque **falso** (else)
`L3`	Etiqueta	Punto de **reunión** (end)
`L4`	Etiqueta	Mid-label del OR: separa evaluación de ambos lados
`if t1 goto L1`	Salto cond.	Cortocircuito: si LEFT es true, va directo al bloque if
`goto L4`	Salto incond.	LEFT fue false, continúa evaluando RIGHT en L4
`if t2 goto L1`	Salto cond.	Si RIGHT es true, va al bloque verdadero
`goto L2`	Salto incond.	Ambos lados fueron false, va al else

Flujo de ejecución (runtime) – Caso 2

1. `t1 = 8 < 7` → `t1 = 0` (false)

2. `if t1 goto L1` → NO salta (LEFT fue false, cortocircuito no aplica)
3. `goto L4` → evalúa el lado derecho
4. `t2 = 9 > 3` → `t2 = 1` (true)
5. `if t2 goto L1` → salta a L1
6. `print "Verdadero"` ✓
7. `goto L3` → fin

Captura 2 – Ejecución `php index.php` (Caso 2)

```
============================================================
CASO 2 — if-else con condición OR (||) y cortocircuito
Entrada:
    if 8 < 7 || 9 > 3 {
        print("Verdadero")
    } else {
        print("Falso")
    }
============================================================

========== Código de 3 Direcciones (C3D) ==========

    t1 = 8 < 7
    if t1 goto L1
    goto L4
L4:
    t2 = 9 > 3
    if t2 goto L1
    goto L2
L1:
    print "Verdadero"
    goto L3
L2:
    print "Falso"
L3:

=====================================================
```

Parte 2 – Validación en el Playground

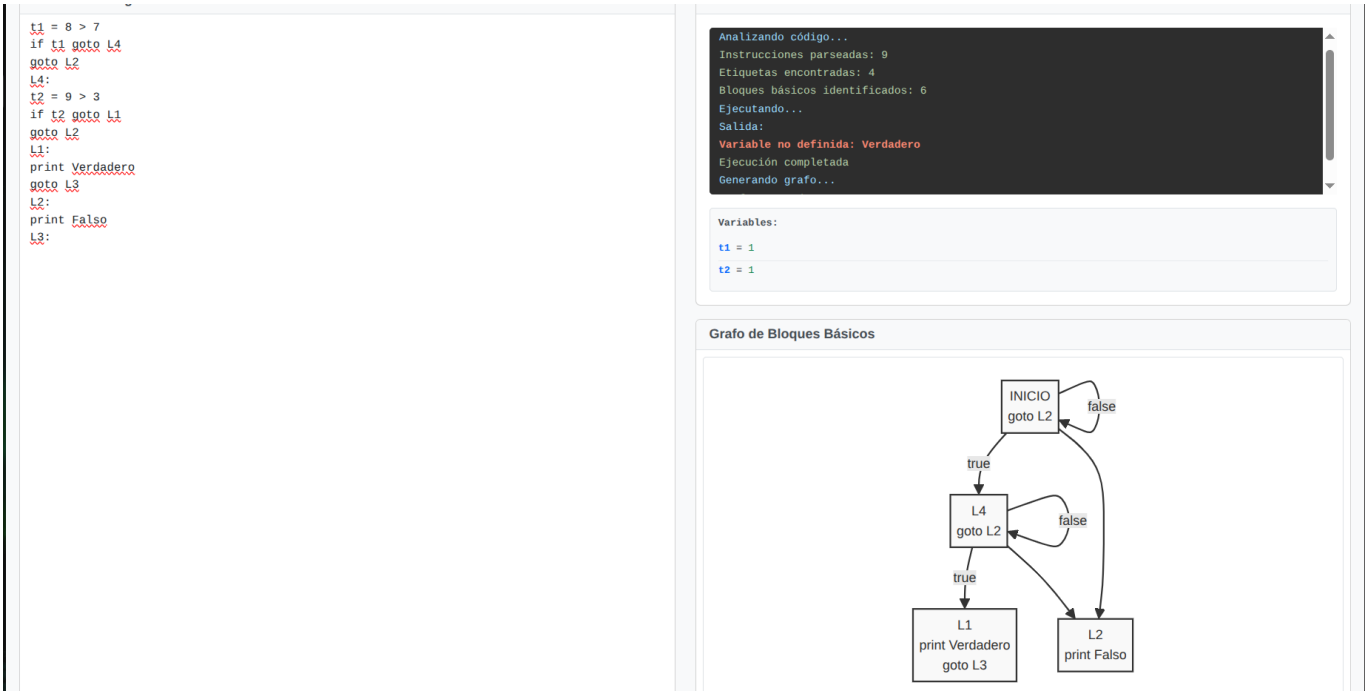
Playground utilizado: **https://diegocali.github.io/simple-c3d-playground/**

> Nota: el playground no soporta strings entre comillas en `print`, por lo que se omiten las comillas únicamente para la validación visual. El C3D generado por la gramática es el correcto (`print "Verdadero"`).

Caso 1 – C3D ingresado en el Playground (AND)

```
t1 = 8 > 7
if t1 goto L4
goto L2
L4:
t2 = 9 > 3
if t2 goto L1
goto L2
L1:
print Verdadero
goto L3
L2:
print Falso
L3:
```

Captura 3 – Playground Caso 1 (AND)



La captura muestra:

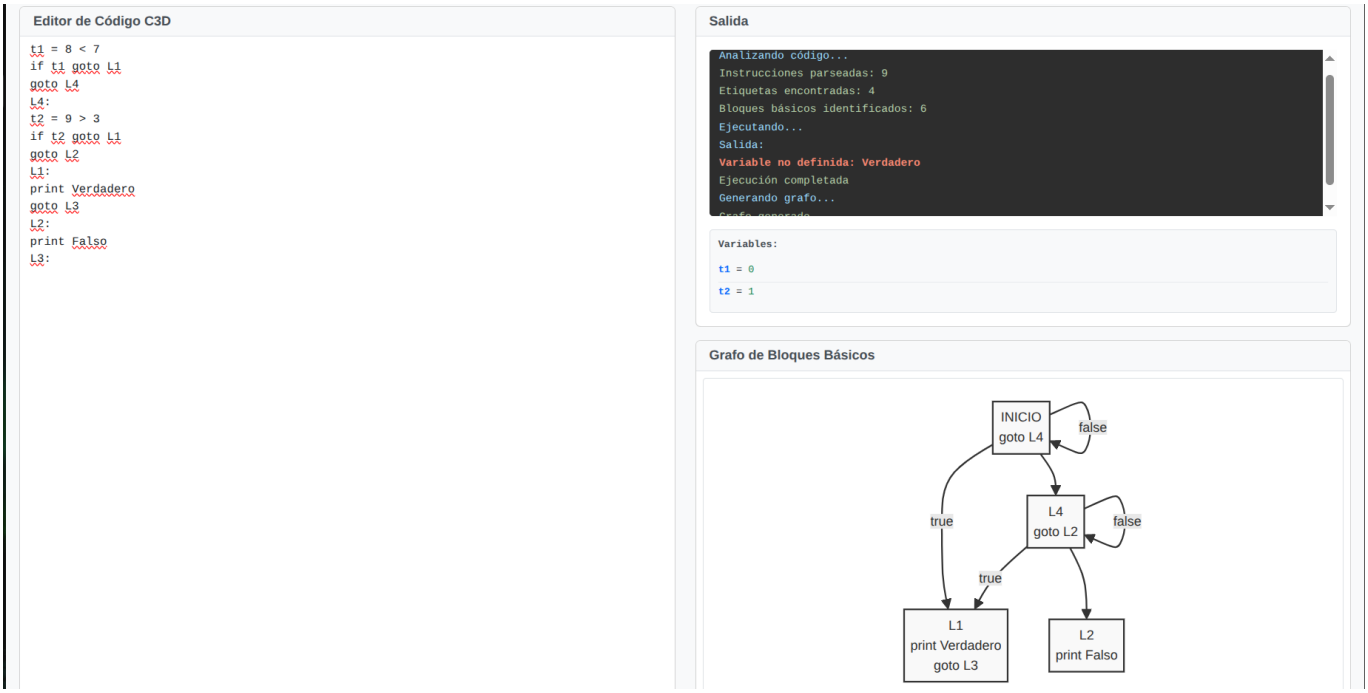
- El C3D del if-else AND en el editor
- Los **bloques básicos**: INICIO, `L4`, `L1`, `L2`, `L3`
- Las **ramas** del grafo: cortocircuito hacia `L2` si `t1=0`, mid-label `L4` si `t1=1`
- Los **temporales**: `t1 = 1`, `t2 = 1`

Caso 2 – C3D ingresado en el Playground (OR)

```
t1 = 8 < 7
if t1 goto L1
goto L4
```

```
L4:
t2 = 9 > 3
if t2 goto L1
goto L2
L1:
print Verdadero
goto L3
L2:
print Falso
L3:
```

Captura 4 – Playground Caso 2 (OR)



La captura muestra:

- El C3D del if-else OR en el editor
- Los **bloques básicos**: INICIO, `L4`, `L1`, `L2`, `L3`
- Las **ramas** del grafo: cortocircuito hacia `L1` si `t1=1`, mid-label `L4` si `t1=0`
- Los **temporales**: `t1 = 0`, `t2 = 1`